

Contribuindo com o ContentFlow

Obrigado por contribuir com o **ContentFlow**.

Este documento define as regras, padrões e processos utilizados no desenvolvimento do projeto. O objetivo é manter o código organizado, facilitar o trabalho em equipe, preservar o histórico de desenvolvimento e garantir que todas as alterações sejam revisadas antes de serem incorporadas ao projeto principal.

Importante: Este projeto utiliza um fluxo baseado em branches e Pull Requests. Alterações diretas na branch `main` não são permitidas.

1. Objetivo

O ContentFlow é um projeto desenvolvido para a disciplina de **Programação para Dispositivos Móveis**, sendo uma evolução de um projeto web previamente desenvolvido.

A versão mobile será desenvolvida seguindo uma arquitetura composta por:

- Aplicativo mobile;
- API REST;
- Banco de dados relacional PostgreSQL;
- Integração com funcionalidades do dispositivo;
- Documentação técnica;
- Controle de versão utilizando Git e GitHub.

O desenvolvimento será realizado em dupla e deverá manter um histórico de commits que represente a evolução real do projeto.

2. Organização do Git

O projeto utiliza três níveis principais de branches:

```
main
|
└─ develop
    |
    ├── feature/*
    ├── fix/*
    ├── refactor/*
    ├── docs/*
    └─ chore/*
```

2.1 main

A branch `main` representa a versão **estável** do projeto.

Ela deve conter somente versões consideradas prontas para apresentação ou entrega.

Regras

- Não realizar commits diretamente na `main`.
 - Não realizar `push` diretamente na `main`.
 - Alterações devem chegar através de Pull Requests.
 - A branch deve permanecer funcional.
 - Commits devem possuir uma versão revisada e estável.
-

2.2 develop

A branch `develop` representa a versão de **desenvolvimento integrada** do projeto.

É nela que as funcionalidades aprovadas são reunidas antes de chegarem à `main`.

Regras

- Não realizar commits diretamente na `develop`, sempre que possível.
 - Novas funcionalidades devem partir de `develop`.
 - Correções devem partir de `develop`.
 - Alterações devem ser integradas através de Pull Requests.
 - A branch deve permanecer executável.
-

3. Branches de trabalho

Cada tarefa deve possuir sua própria branch.

O nome da branch deve indicar claramente o tipo e o objetivo da alteração.

Formato

tipo/descricao

Tipos permitidos

Tipo	Utilização
<code>feature/</code>	Nova funcionalidade
<code>fix/</code>	Correção de erro

Tipo	Utilização
<code>refactor/</code>	Refatoração sem alteração de comportamento
<code>docs/</code>	Documentação
<code>chore/</code>	Configurações e tarefas auxiliares
<code>test/</code>	Criação ou alteração de testes

Exemplos

```
feature/login
feature/criar-post
feature/listar-publicacoes
feature/notificacoes
fix/erro-autenticacao
fix/validacao-formulario
refactor/estrutura-api
docs/atualizar-readme
chore/configurar-eslint
test/teste-login
```

Regras de nomenclatura

- Utilizar letras minúsculas.
- Utilizar `-` para separar palavras.
- Não utilizar espaços.
- Não utilizar acentos.
- Evitar nomes genéricos como:

```
teste
nova-branch
coisa
alteracoes
final
final2
agora-vai
```

4. Fluxo de desenvolvimento

Todo desenvolvimento deve seguir o seguinte fluxo:

```
develop
↓
Criar branch
↓
```

```
Desenvolver
↓
Commit
↓
Push
↓
Pull Request
↓
Code Review
↓
Correções, se necessário
↓
Aprovação
↓
Merge em develop
↓
Testes
↓
Merge em main
```

5. Criando uma nova branch

Antes de iniciar uma nova tarefa, atualizar a branch `develop`.

```
git checkout develop
git pull origin develop
```

Criar a branch:

```
git checkout -b feature/nome-da-funcionalidade
```

Exemplo:

```
git checkout -b feature/login
```

Após criar a branch, todo o desenvolvimento daquela tarefa deve acontecer nela.

6. Commits

Os commits devem representar a evolução real do projeto.

Não serão utilizados commits genéricos que não expliquem o que foi realizado.

6.1 Padrão

Será utilizado o padrão:

tipo: descrição

Tipos

Tipo	Significado
feat	Nova funcionalidade
fix	Correção de erro
refactor	Refatoração
docs	Documentação
style	Alterações de formatação/estilo
test	Testes
chore	Configurações/manutenção

Exemplos

```
feat: criar tela de login
feat: adicionar validação do formulário
feat: implementar endpoint de autenticação
fix: corrigir validação da senha
docs: atualizar documentação da API
refactor: reorganizar serviço de usuários
test: adicionar testes para autenticação
chore: configurar variáveis de ambiente
```

7. Regras para commits

Cada commit deve representar uma alteração lógica.

Evitar

```
fiz tudo
mudanças
teste
arrumei
```

```
final  
agora foi
```

Preferir

```
feat: criar tela de cadastro  
feat: adicionar validação de email  
feat: implementar criação de usuário  
fix: corrigir validação de senha
```

7.1 Commits pequenos

Sempre que possível, dividir o desenvolvimento em pequenas etapas.

Por exemplo, uma funcionalidade de login pode possuir:

```
feat: criar interface de login  
feat: adicionar validação do formulário  
feat: criar endpoint de autenticação  
feat: integrar login com API  
fix: corrigir tratamento de erro no login
```

Isso permite que o histórico do Git represente a evolução real do projeto.

8. Push

Após realizar os commits, enviar a branch para o GitHub:

```
git push -u origin nome-da-branch
```

Exemplo:

```
git push -u origin feature/login
```

9. Pull Request

Toda funcionalidade deve ser integrada através de um Pull Request.

O Pull Request deve ser aberto da branch de trabalho para:

```
feature/* → develop
```

Correções:

```
fix/* → develop
```

Documentação:

```
docs/* → develop
```

10. Estrutura do Pull Request

Todo Pull Request deve conter:

Título

O título deve ser objetivo.

Exemplo:

```
feat: implementar autenticação de usuários
```

Descrição

A descrição deve informar:

- O que foi desenvolvido;
- Qual problema foi resolvido;
- Quais arquivos ou áreas foram alterados;
- Como testar;
- Se existe alguma observação importante.

Exemplo

Descrição

Implementa o fluxo inicial de autenticação do usuário.

Alterações

- Criada tela de login;
- Criada validação dos campos;
- Criado endpoint de autenticação;

- Integrado aplicativo com a API.

Como testar

1. Executar a API;
2. Executar o aplicativo;
3. Acessar a tela de login;
4. Informar email e senha;
5. Verificar o retorno da autenticação.

Checklist

- [x] Código compilando
- [x] Funcionalidade testada
- [x] Sem erros conhecidos
- [x] Documentação atualizada

11. Code Review

Antes do merge, o código deve ser revisado pelo outro integrante da dupla.

O revisor deve verificar:

- Se a funcionalidade corresponde ao objetivo da tarefa;
- Se o código está organizado;
- Se não existem erros evidentes;
- Se a alteração não quebrou funcionalidades existentes;
- Se os nomes utilizados são claros;
- Se não existem informações sensíveis no código;
- Se a documentação necessária foi atualizada;
- Se a branch contém somente alterações relacionadas à tarefa.

12. Aprovação

Um Pull Request somente poderá ser integrado quando:

- A funcionalidade estiver funcionando;
- O código tiver sido revisado;
- Os problemas encontrados forem corrigidos;
- O projeto continuar executando normalmente;
- O outro integrante aprovar a alteração.

Para uma equipe de duas pessoas, recomenda-se que **quem desenvolveu a funcionalidade não seja a única pessoa responsável pela aprovação do próprio código.**

13. Merge

Após a aprovação:

```
feature/login
  ↓
develop
```

O merge deverá ser realizado pelo GitHub.

Sempre que possível, manter a branch de desenvolvimento organizada e evitar merges desnecessários.

Após uma funcionalidade ser integrada:

```
git checkout develop
git pull origin develop
```

14. Main

A `main` representa uma versão estável do projeto.

A atualização da `main` deverá ocorrer somente quando uma versão estiver pronta para apresentação, avaliação ou entrega.

Fluxo:

```
feature
  ↓
develop
  ↓
testes
  ↓
main
```

15. Conflitos

Caso ocorram conflitos durante a integração:

1. Identificar os arquivos conflitantes;
2. Analisar as alterações de ambas as branches;
3. Resolver o conflito manualmente;

4. Executar os testes;
5. Confirmar que nenhuma funcionalidade foi perdida;
6. Realizar o commit da resolução;
7. Continuar o processo de integração.

Nunca resolver conflitos simplesmente escolhendo todas as alterações de uma branch sem verificar o conteúdo.

16. Regras de proteção

A branch `main` deverá possuir, sempre que possível, as seguintes proteções no GitHub:

- Bloqueio de push direto;
- Alterações somente através de Pull Request;
- Revisão obrigatória;
- Histórico de alterações preservado;
- Bloqueio de exclusão da branch.

A branch `develop` também deverá ser protegida sempre que a configuração do projeto permitir.

17. Variáveis de ambiente

Informações sensíveis nunca devem ser adicionadas ao Git.

Exemplos:

```
.env
.env.local
.env.production
```

Não devem ser enviados:

- Senhas;
- Tokens;
- Chaves de API;
- Credenciais de banco;
- Segredos JWT;
- Informações privadas.

O projeto deverá disponibilizar um arquivo de exemplo quando necessário:

```
.env.example
```

Exemplo:

```
DATABASE_URL=  
JWT_SECRET=  
API_URL=
```

O arquivo `.env.example` deve conter apenas os nomes das variáveis, nunca os valores reais.

18. Organização das tarefas

Cada funcionalidade deve possuir uma tarefa claramente definida.

Antes de iniciar uma implementação, deve ser possível responder:

- O que será desenvolvido?
- Qual problema será resolvido?
- Qual parte do sistema será alterada?
- Quem será responsável?
- Como saberemos que está funcionando?

Sempre que possível, as tarefas devem ser registradas no GitHub Issues.

Exemplo:

```
#12 Implementar autenticação de usuários
```

A branch pode então ser:

```
feature/autenticacao-usuarios
```

19. Relação entre Issue, Branch e Pull Request

Sempre que possível, manter a seguinte relação:

```
Issue  
↓  
Branch  
↓  
Commits  
↓  
Pull Request  
↓  
Review
```

↓
Merge

Exemplo:

```
Issue #12
Implementar login
  ↓
feature/login
  ↓
feat: criar tela de login
feat: integrar autenticação
  ↓
Pull Request #15
  ↓
Code Review
  ↓
develop
```

20. Documentação

Toda alteração que modificar significativamente o funcionamento do sistema deve avaliar se a documentação precisa ser atualizada.

Podem ser atualizados:

- `README.md`;
- Diagramas;
- Documentação da API;
- Casos de uso;
- Diagrama de classes;
- Documentação do banco;
- Instruções de instalação;
- Variáveis de ambiente.

A documentação deve acompanhar a evolução do sistema.

21. Testes antes do Pull Request

Antes de abrir um Pull Request, o responsável deve verificar:

```
[ ] O projeto inicia normalmente
[ ] A funcionalidade foi testada
[ ] Não existem erros conhecidos
```

- [] Não existem credenciais no código
- [] O código está organizado
- [] A documentação necessária foi atualizada
- [] Os commits possuem mensagens claras
- [] A branch está atualizada com develop

22. Atualizando a branch antes do Pull Request

Antes de abrir ou finalizar um Pull Request, atualizar a branch:

```
git checkout develop  
git pull origin develop
```

Depois retornar para sua branch:

```
git checkout feature/minha-feature
```

E integrar as alterações mais recentes de acordo com o fluxo definido pela equipe.

Após resolver possíveis conflitos, executar novamente os testes.

23. O que não fazer

Não realizar:

- ✗ Commit direto na main
- ✗ Push direto na main
- ✗ Commit com mensagem genérica
- ✗ Enviar senha ou token para o GitHub
- ✗ Fazer uma grande alteração sem commits intermediários
- ✗ Misturar várias funcionalidades diferentes em uma branch
- ✗ Apagar histórico de commits sem necessidade
- ✗ Fazer merge sem revisão
- ✗ Ignorar conflitos
- ✗ Enviar código que não foi testado

24. Histórico de desenvolvimento

O histórico do GitHub faz parte da documentação do projeto.

Os commits devem representar o desenvolvimento realizado durante o período da disciplina.

Portanto, não é recomendado desenvolver todo o projeto localmente e realizar vários commits somente no final.

O histórico deve demonstrar a evolução natural do sistema:

```
Configuração
  ↓
Interface
  ↓
Funcionalidades
  ↓
API
  ↓
Banco de dados
  ↓
Integrações
  ↓
Correções
  ↓
Refatorações
  ↓
Versão final
```

25. Responsabilidade dos integrantes

Os dois integrantes são responsáveis pela qualidade do projeto.

Cada integrante deve:

- Desenvolver suas tarefas;
- Manter suas branches organizadas;
- Realizar commits frequentes e significativos;
- Revisar Pull Requests;
- Comunicar problemas;
- Testar alterações;
- Manter a documentação atualizada;
- Evitar alterações que possam prejudicar o trabalho do outro integrante.

26. Fluxo resumido

Para uma nova funcionalidade:

```
git checkout develop
git pull origin develop

git checkout -b feature/minha-feature

# desenvolvimento

git add .
git commit -m "feat: implementar minha feature"

git push -u origin feature/minha-feature
```

Depois:

```
GitHub
  ↓
Pull Request
  ↓
Code Review
  ↓
Aprovação
  ↓
Merge → develop
```

Quando a versão estiver pronta:

```
develop
  ↓
Pull Request
  ↓
main
```

27. Regra principal

Nenhuma alteração deve entrar no projeto principal sem que seu desenvolvimento possa ser compreendido através do histórico do Git.

O GitHub não será utilizado apenas como local para armazenar o código, mas como ferramenta para registrar a evolução, colaboração e organização do projeto.

28. Checklist final de contribuição

Antes de considerar uma tarefa concluída:

- ☐ Issue definida
- ☐ Branch criada a partir de develop
- ☐ Desenvolvimento realizado na branch
- ☐ Commits pequenos e descritivos
- ☐ Código testado
- ☐ Documentação atualizada, quando necessário
- ☐ Push realizado
- ☐ Pull Request aberto
- ☐ Code Review realizado
- ☐ Alterações solicitadas corrigidas
- ☐ Pull Request aprovado
- ☐ Merge realizado em develop
- ☐ Branch de trabalho encerrada, quando não for mais necessária

29. Conclusão

O objetivo deste processo é garantir que o ContentFlow possua não apenas um código funcional, mas também um **histórico de desenvolvimento organizado, rastreável e colaborativo**.

Todas as contribuições devem respeitar este documento.

Em caso de dúvida sobre o fluxo de desenvolvimento, a decisão deverá priorizar:

1. Integridade do código;
2. Clareza do histórico;
3. Revisão das alterações;
4. Organização do projeto;
5. Evolução contínua do sistema.